

System Architecture Design

Persistent World RTS (Supremacy 1914 Clone)

This document details the scalable backend architecture for a Persistent World Real-Time Strategy (PWRTS) game. The system is designed to support thousands of concurrent matches with varying player counts, utilizing a vector-based province graph, mid-travel interceptions, and an optimized event-driven state engine.

1. Core Architectural Philosophy

The architecture eschews the traditional "global tick" loop found in fast-paced RTS games (which causes extreme CPU spikes at scale) in favor of a **Hybrid Discrete Event Simulation (DES)**.

- **Event-Driven Micro-Ticks:** Actions (e.g., unit arrivals, interceptions) are pre-calculated and scheduled in a Priority Queue (Timeline Min-Heap). The engine only processes exactly when an event is scheduled to occur.
- **Macro-Pulse:** A slow-running pulse (e.g., every 15 minutes) processes bulk calculations like resource production, morale decay, and maintenance costs, distributing the load evenly over time.

2. Technology Stack

Component	Technology	Rationale
Core Engine	Go (Golang)	Lightweight goroutines handle thousands of concurrent match loops perfectly. High performance for graph traversal and vector math.
Messaging / RPC	NATS JetStream gRPC	NATS provides high-throughput pub/sub for real-time state broadcasts. gRPC powers strict, strongly-typed internal service communication.
Hot State	Redis	Stores the spatial index, active unit positions, and fast-access match state. Allows for distributed locking (Redlock) to prevent race conditions.
Cold Storage	PostgreSQL	ACID-compliant persistence for player accounts, match metadata, map definitions, and periodic match-state snapshots.

3. High-Level System Design

Component Roles

- **Stateless WebSocket Gateway:** Manages persistent user connections. Subscribes to specific NATS subjects dynamically (e.g., `match.123.player.456`). Allows the underlying game engine to restart without dropping client connections.
- **Game Engine Nodes:** Stateful instances written in Go. Each node handles a shard of active matches. When a player issues a command, the engine recalculates the timeline and inserts a future event.
- **Visibility Service:** Listens to game state mutations and calculates Line of Sight (LoS). It ensures that "Fog of War" masks are applied before emitting updates to the gateway, maintaining authoritative hidden state.

4. Vector-Based Map & Movement Engine

The map is mathematically represented as a **Graph** $G = (V, E)$, entirely decoupling the logical behavior from visual representation (SVG/Phaser on the frontend).

Continuous Space Interception Math

Because units can intercept each other mid-travel along an edge, we represent movement as a vector function of time:

$$P(t) = P_{start} + v \rightarrow (t - t_{start})$$

Interception occurs when the distance between Unit A and Unit B is less than or equal to their combat radius R :

$$\|P_A(t) - P_B(t)\| \leq R_A + R_B$$

When a movement command is issued, the engine pre-calculates the **Time of Impact (Tol)** using linear kinematics and injects a `COMBAT_START` event into the Priority Queue for that specific timestamp.

Handling "Stale" Events

A major challenge in DES models occurs when pre-calculated events are invalidated (e.g., a unit is ordered to retreat before the impact time). We solve this via **Validation Signatures**:

- Every scheduled event carries a `version_id` linking to the unit's command state at the time of scheduling.

- When the timeline pops the COMBAT_START event, the engine performs a *O(1)* validation check:
"Does the unit's current command version match the event's version?"
- If false, the event is silently dropped as a "stale" state.

5. Fog of War & Spatial Partitioning

Calculating visibility in an $N \times N$ entity environment scales poorly. The engine avoids this via Graph-based Spatial Partitioning:

- **Edge-Unit Mapping:** Redis maintains a lookup map of `Map<EdgeID, List<UnitID>>`. Collision checks are bounded strictly to units traversing the same or directly adjacent edges.
- **Radial Graph Search:** Visual distance is not calculated by 2D grids, but by traversing graph edges. The engine performs parallelized graph walks outward from a unit's current position until the cumulative vector distance exceeds the unit's *view_radius*.

6. Data Consistency & State Write-Back

To avoid hammering the PostgreSQL database, the architecture employs a Write-Back caching strategy:

- **Mutations:** All real-time commands (moves, combat) mutate the "Hot State" residing in Redis. Redis transactions (or Lua scripts) guarantee atomic updates for scenarios like simultaneous province attacks.
- **Snapshots:** A dedicated sidecar worker periodically (e.g., every 5 minutes) triggers a snapshot of the Redis state and streams it asynchronously to PostgreSQL.
- **Recovery:** In the event of a Game Engine crash, a new node spins up, pulls the latest exact state from Redis, restores the Timeline Min-Heap, and resumes the match seamlessly.